23CSE452 BUSINESS ANALYTICS

Capstone Project: Final Report

Mobile App Update Impact and Failure Detection

Complaint mining, forecasting and release monitoring for 11 Indian consumer apps

Team 10

Member	Register number
Aditya Monish Kumar K	CB.SC.U4CSE23103
Akshay KS	CB.SC.U4CSE23104
Kanishka D	CB.SC.U4CSE23155
Harshini Vennela	CB.SC.U4CSE23455
Regella Krishna Saketh	CB.SC.U4CSE23649

Programme: B.Tech. Computer Science and Engineering, Semester 7

Faculty: Rohini S.

Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore

Academic year 2026-2027 · Submission date: 10/11/2026

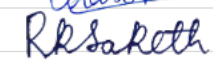
Live dashboard: https://team10-app-reviews.streamlit.app/

Repository: https://github.com/Adithya-Monish-Kumar-K/ba-capstone-app-reviews

Project board: https://github.com/users/Adithya-Monish-Kumar-K/projects/4

Declaration

We declare that the dataset, analysis and software described in this report are the work of Team 10 for 23CSE452 Business Analytics. The reviews were collected by us from the Google Play Store, and no pre-built Kaggle, UCI or similar dataset was used. Borrowed ideas, methods and software are cited. Generative-AI assistance and verification are disclosed in Section 8.3.

Name	Register number	Signature
Aditya Monish Kumar K	CB.SC.U4CSE23103	
Akshay KS	CB.SC.U4CSE23104	
Kanishka D	CB.SC.U4CSE23155	
Harshini Vennela	CB.SC.U4CSE23455	
Regella Krishna Saketh	CB.SC.U4CSE23649	

Acknowledgement

We thank Rohini S. for guidance and feedback across both reviews, and we acknowledge the maintainers of the open-source tools used in this project.

Abstract

Product and support teams need a practical way to identify complaints and investigate harmful app updates in large review streams. We collected Google Play reviews posted from 1 April to 20 September 2026 through the English-language India storefront feed for 11 apps across food and grocery, shopping and payments. Cleaning retained 1,137,987 reviews and 29 variables. Exploratory analysis found strong differences between apps and a feed gap that removed positive reviews. A class-weighted Random Forest identified low-rating reviews with PR-AUC 0.8735 and recall 0.8701; a training-only preprocessing audit gave PR-AUC 0.8739. Two Unit 3 methods extended the analysis. Non-negative matrix factorisation assigned 167,198 sufficiently long complaints to 17 themes and outperformed LDA on held-out coherence in all three domains. ARIMA/SARIMAX forecasts achieved mean MASE 0.934 and 93.9% coverage for nominal 95% intervals. Release tests flagged four of 205 testable releases, one of them with corroborating app-level evidence. The methods feed a six-signal priority rule, 65 recommendations and an interactive Streamlit application. The findings support complaint triage and release monitoring; rating proxies, collection bias, evaluation limitations and the observational design constrain how they should be read.

Executive Summary

Product managers and support leads need to know which reviews deserve attention, what users are reporting, and whether a new release coincides with a worse experience. This project brings those decisions into one workflow over 1.14 million reviews from 11 apps and 173 days.

The main operational finding is that complaints which name an issue usually concern service: 91.3% of tagged low-rating reviews mention delivery, orders, returns, support or pricing. Topic models add themes beyond keyword rules. Refunds and cancellations lead food and grocery complaints (34.7%), returns and product quality lead shopping (25.4%), and payment, UPI and bank-account problems lead payments (48.6%). These shares describe the topic-assigned corpus, not every customer or every low-rating review.

The Random Forest classifier flags 23.0% of reviews while recovering 87.0% of low-rating reviews. Forecasts beat the same-weekday-last-week baseline on MAE for 10 apps, and eight apps have MASE below 1. Overall 95% interval coverage is 93.9%, although shopping coverage is uneven. Release tests find no clear change in 196 releases, improvement in five and deterioration signals in four. Only Paytm 10.84.5 has corroborating app-level count evidence, and association is not proof of causation.

Under explicit assumptions (10 seconds per review, Rs 250 per analyst hour, 30 operating days a month and 8.7 maintenance hours a month), selective triage could save about 14.1 reviewer-hours a day, or Rs 103,300 a month net. These are scenario estimates, not measured savings. A separate Paytm illustration implies about 182 extra low-rating reviews a week if the observed 18-day excess persisted; avoiding them would need a successful fix.

We recommend prioritising Swiggy refund and cancellation operations, Amazon customer care, Flipkart volume alerts and an investigation of Paytm 10.84.5, and starting triage with human review in Payments. The application and analyses are implemented; five of the six objectives are met (O4 as an exploratory screen), and the dashboard objective is only partly met because hosted response time was not measured. The roadmap in Section 9 assigns owners and success measures.

Table of Contents

Abbreviations ..... **v**
1. Introduction, Business Objectives and Literature Review **1**
1.1 Context, scope and analytical questions 1
1.2 Related work and the research gap 1
2. Dataset Collection, Validation and Preparation **2**
2.1 Source coverage and collection challenges 2
2.2 Data quality, privacy and ethics 3
2.3 Transformations and analysis-ready inputs 3
3. Exploratory Data Analysis and Visualization (Unit 1) **4**
4. Feature Engineering and Predictive Modelling (Unit 2) **5**
4.1 Target, features and evaluation protocol 5
4.2 Leakage audit, errors and subgroup performance 6
5. Advanced Analytics (Unit 3): Two Selected Methods **7**
5.1 Method 1: text mining with NMF topics and sentiment 7
5.2 Method 2: time-series forecasting and release monitoring 9
6. Comparative Analysis and Business Impact **13**
6.1 Complementary methods and combined findings 13
6.2 Comparison with existing studies 13
6.3 KPI-linked business value, priorities and sensitivity scenarios 14
7. Interactive Application, Figure Index and Reproducibility **16**
7.1 Application architecture and working interactions 16
7.2 Figure index and reproduction map 16
7.3 Code and environment 17
8. End-to-End Validation, Traceability and Contributions **17**
8.1 Objective outcomes and validation scope 17
8.2 Project tracking and individual contributions 17
8.3 AI use and originality declaration 19
9. Conclusion, Recommendations and Roadmap **19**
References **20**
Appendix A: Data Dictionary and Validation **21**
Appendix B: Supplementary Evaluation **23**
Appendix C: Repository Environment and Run Instructions **25**
Appendix D: Tracking Evidence **27**

List of Figures

Fig. 1. End-to-end workflow....1
Fig. 2. Within-domain app comparisons....4
Fig. 3. Positive-review feed gap....5
Fig. 4. Precision–recall and RF feature importance....6
Fig. 5. NMF versus LDA comparison....7
Fig. 6. Leading terms in 17 NMF topics....8
Fig. 7. Rolling backtests for all 11 apps....11
Fig. 8. Paytm release-window counts....12
Fig. 9. Six warning signs and app priorities....15
Fig. 10. Overview and Paytm Forecasting screenshots....16
Fig. D1. Review 2 project board....27

List of Tables

Table 1. Objectives and success criteria....1
Table 2. Final-data sample....2
Table 3. Cleaning counts....3
Table 4. Dataset mapping to Units 1–3....3
Table 5. Classifier comparison....6
Table 6. Training-only preprocessing audit....6
Table 7. Original-experiment confusion counts....6
Table 8. RF results by domain....7
Table 9. Illustrative complaint texts....9
Table 10. Selected forecast orders....9
Table 11. Overall forecast accuracy....10
Table 12. Per-app forecast errors and coverage....10
Table 13. Flagged releases....12
Table 14. Comparison of Unit 3 methods....13
Table 15. Comparison with prior studies....13
Table 16. Business sensitivity scenarios....14
Table 17. Figure reproduction map....16
Table 18. Objective-to-evidence traceability....17
Table 19. Individual contributions....18
Table 20. Roadmap and success checks....19
Table A1. Complete dictionary of 29 fields....21
Table A2. Recorded validation checks....22
Table B1. Bootstrap confidence intervals....23
Table B2. Feature ablation....23
Table B3. RF performance for all 11 apps....23
Table B4. Complete topic shares and sentiment....24
Table C1. Dashboard user guide and filter scope....26

Abbreviations

ADF: augmented Dickey-Fuller; AICc: corrected Akaike criterion; ARIMA/SARIMAX: (seasonal) autoregressive integrated moving average, optionally with exogenous inputs; BH/FDR: Benjamini-Hochberg / false discovery rate; EDA: exploratory data analysis; LR/RF: Logistic Regression / Random Forest; MAE/RMSE: mean absolute / root mean squared error; MASE: mean absolute scaled error; NMF/LDA: non-negative matrix factorisation / latent Dirichlet allocation; NPMI: normalised pointwise mutual information; PR-AUC: precision-recall area, reported as average precision; SVD: singular value decomposition; TF-IDF: term frequency-inverse document frequency; UPI: Unified Payments Interface; VADER: lexicon-based sentiment scorer.

1. Introduction, Business Objectives and Literature Review

1.1 Context, scope and analytical questions

Indian consumer apps receive between 117 and 1,539 written reviews per app per day in this dataset. Product managers need release signals, support leads need a triage queue and a capacity forecast, and quality and operations teams need specific issues to investigate. The project therefore asks five questions: Q1, which reviews resemble dissatisfied users; Q2, what users complain about; Q3, how complaint volume changes and which days are unusual; Q4, whether releases coincide with deterioration; and Q5, what to fix first.

The scope covers Swiggy, Zomato, Blinkit and Domino's (food and grocery); Myntra, Flipkart, Amazon and Meesho (shopping); and Paytm, PhonePe and Google Pay (payments), observed from 1 April to 20 September 2026. A complaint means a 1-2 star review, which is a dissatisfaction proxy rather than a verified software defect. iOS, internal business outcomes, a full annual cycle and causal attribution are outside the scope.

Table 1. Business objectives and success criteria

Objective	Measure and target	Baseline / route
O1 Triage	PR-AUC >= 0.80; recall >= 85%	Base rate 0.195; Unit 2
O2 Theme complaints	>= 90% of eligible, sufficiently long complaints; NMF coherence above LDA	57.1% keyword coverage in the themed corpus; text mining
O3 Forecast counts	MASE < 1 for >= 8 apps; pooled 95% coverage 90-97%	Seasonal naive mean MASE 1.080
O4 Screen releases	Test all eligible releases with BH at 5% FDR	No prior release screen
O5 Prioritise	Traceable priorities for 11/11 apps	Combine Units 1-3
O6 Deliver tool	Accessible or local dashboard; response target < 2 s	Functionality and measured test states

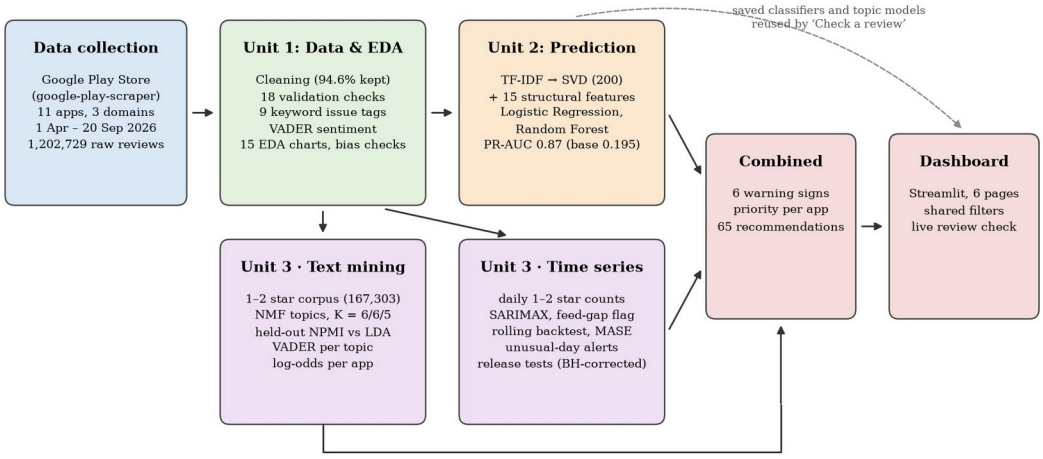


Fig. 1. End-to-end workflow across Units 1 to 3 and the application

1.2 Related work and the research gap

Pagano and Maalej [18] studied release-related feedback patterns in more than one million App Store reviews. AR-Miner [5] filtered informative reviews and grouped them for developers, and Guzman

and Maalej [8] analysed feature-level sentiment. Maalej and Nabil [17] classified review types using text, metadata and sentiment, while IDEA [6] tracked emerging topics between versions, showing that review analysis need not treat feedback as a static collection. Khalid et al. [14] built a taxonomy of the complaints behind low-rated iOS reviews, Palomba et al. [19] linked reviews to the code changes that addressed them and reported higher ratings for apps that acted on user requests, and Villarroel et al. [25] clustered and prioritised reviews for release planning.

What remains is an integrated, domain-specific workflow that links complaint themes, volume expectations, release screening and priorities. We combine these outputs for Indian consumer apps and explicitly audit a collection artefact. This is a workflow contribution, not a claim of novelty for NMF, ARIMA or classification; Section 6 compares the completed findings with prior work. Association rules were not selected because only about 3.4% of reviews carry more than one issue tag.

2. Dataset Collection, Validation and Preparation

2.1 Source coverage and collection challenges

We used google-play-scraper [13] with language en, country in and newest-first ordering, paging backwards through the common window. The source is the public storefront feed, not a pre-built dataset. These settings do not establish every reviewer's physical location and do not guarantee an exhaustive census, so the data should be read as the reviews the feed returned. Romanised Hindi (Hinglish) remains after the Latin-letter filter. The raw collection has 1,202,729 rows; the final tagged dataset has 1,137,987 rows and 29 variables, with one row per review.

Table 2. Sample rows of the final dataset (6 of 29 columns)

App	Score	Review text	Date	Primary issue	VADER
Swiggy	1	bekar service hai inki	2026-06-07	none	+0.00
Amazon	1	Items never get delivered and get cancelled...	2026-07-23	cancellation_return	-0.23
Paytm	5	Koi reward nahin mila	2026-06-12	none	+0.57
Flipkart	5	flipkart provide order on time	2026-07-26	none	+0.00
Zomato	1	Worst customer care support no customer care number	2026-08-21	customer_support	+0.48

The default most-relevant ordering gave a capped and skewed trial, so collection switched to newest-first. Large pulls used retries and per-app restart handling. From 21 April to 5 May, positive reviews fell by 59-90% for five apps; these rows are retained and flagged. Zomato has a source gap from 23 July 22:00 to 25 July 12:30, so 24 July is fully absent and the adjacent days are partial. Data collection for the project began on 20 September 2026.

2.2 Data quality, privacy and ethics

Table 3. Cleaning counts and retained records

Cleaning step	Removed	Retained
Raw collection	-	1,202,729
Duplicate review IDs	0	1,202,729
Empty or fewer than 3 characters	47,533	1,155,196
No letters	7,420	1,147,776
Fewer than 85% Latin letters	9,789	1,137,987

All 18 validation checks pass: review IDs are unique, ratings and dates are valid, derived fields are consistent and every app maps to one domain. Missing versions affect 152,017 reviews (13.4%); they stay in the data but are not classifier predictors, and all other final columns are complete. Long reviews and heavy-tailed upvotes are retained as plausible observations. Low ratings account for 221,581 reviews (19.47%).

Only public text, opaque IDs and app metadata were kept; reviewer names and profile images were discarded, and IDs support deduplication, not contact. Third-party reviews remain subject to their owners' rights, so academic use does not grant redistribution rights. The DPDP Act 2023 [7] informs our privacy practice but we do not claim it certifies compliance. Raw text may contain identifiers that reviewers volunteered, so public examples need review, and raw files are not asserted to be fully anonymised.

2.3 Transformations and analysis-ready inputs

Overall cleaning adds month and word count, nine keyword issue flags with summary fields, and four VADER scores plus a sentiment label. The language filter identifies script rather than language: Hinglish survives and may be scored poorly by English VADER. Written reviews are self-selected and 0.0-1.4 stars harsher than public app ratings, so conclusions describe this review stream rather than all customers. Classification, topic modelling and forecasting each use preparation suited to the task (Sections 4 and 5).

Table 4. Mapping the final dataset to Units 1 to 3

Fields	Preparation and use
score	Unit 1 distribution; Unit 2 target; Unit 3 complaint definition
content	Keyword tags and VADER; classifier TF-IDF; cleaned topic corpus
review_date, app_version	Chronological validation; daily counts; inferred release events
issue flags, sentiment, length, upvotes	15 structural predictors; issue and sentiment interpretation
Derived Unit 3 inputs	167,303 eligible complaints; 11 daily series; 224 release events

3. Exploratory Data Analysis and Visualization (Unit 1)

The rating distribution is polarised: 66.54% of reviews are 5 stars and 17.30% are 1 star. The median review is two words, compared with 14 words for a 1-star review. Rating and length have a Spearman correlation of -0.391, and rating and VADER negative score a correlation of -0.599. One-star reviews receive 59.21% of helpful votes. Ratings differ by app (chi-square 65,055.1, 40 degrees of freedom, $p < 0.001$), although with this sample size the effect sizes and practical differences matter more than significance.

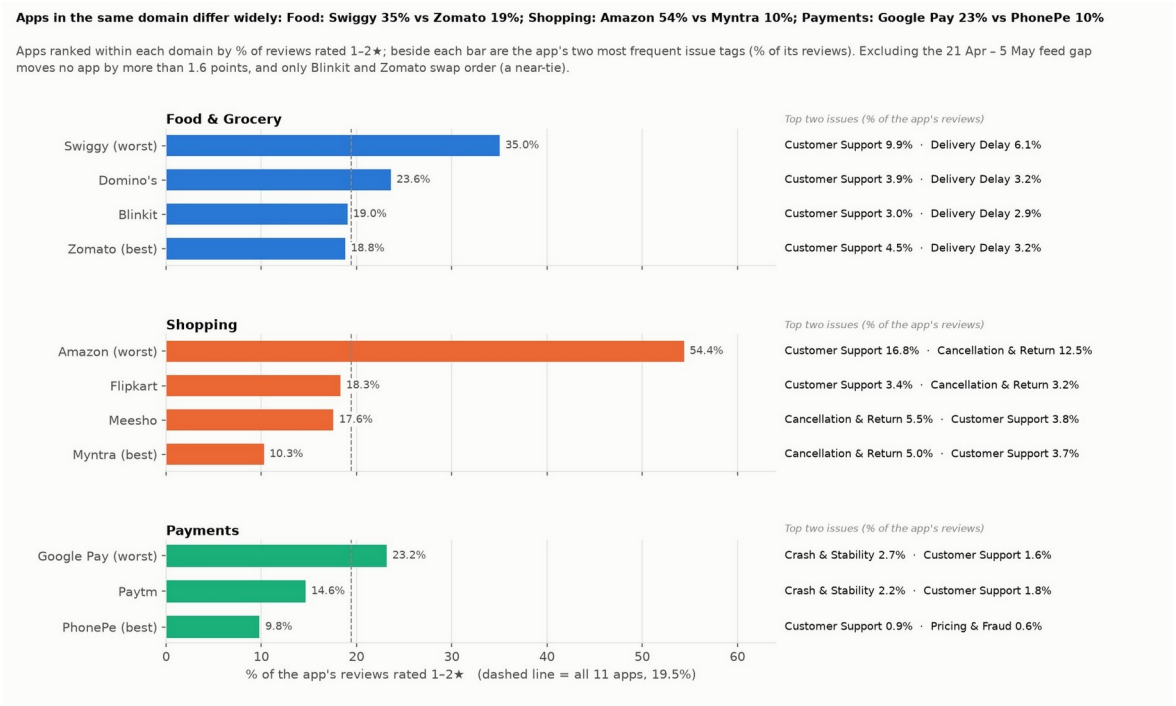


Fig. 2. Within-domain differences across all 11 apps, showing low-rating shares and the two leading tagged issues

Amazon has a 54.4% low-rating share against Myntra's 10.3%, and Swiggy 35.0% against Zomato's 18.8%, so within-domain reporting is more useful than a single pooled ranking. Among tagged low-rating reviews, 91.3% mention service issues; this excludes the 54.3% of low-rating reviews that carry no keyword tag.

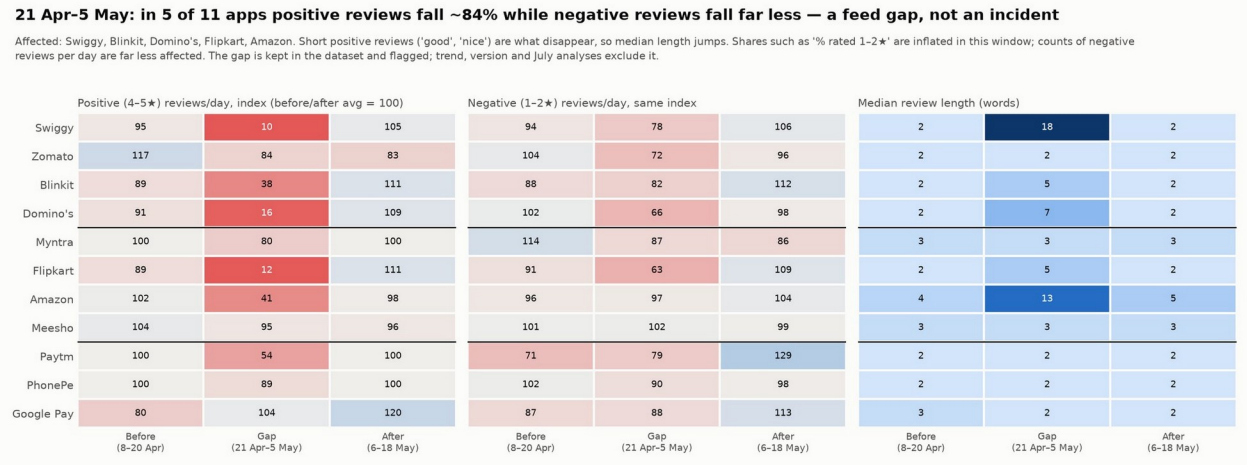


Fig. 3. Positive and negative review volumes and median text length before, during and after the 21 April to 5 May feed gap

For Swiggy, the positive-review index falls to 10 during the gap while the negative-review index is 78 (each relative to the mean of the before and after windows, set to 100), and median review length rises from two to 18 words. Amazon shows indices of 41 and 97. The joint loss of short positive reviews supports treating the period as a collection artefact rather than reading the rating-share shift as product deterioration.

The gap changes the denominator of rating shares, so a jump in low-rating percentage does not show a rise in complaints when positive reviews disappear. Unit 3 therefore forecasts low-rating counts with a gap indicator, and the combined priorities exclude affected app-days when computing complaint shares.

4. Feature Engineering and Predictive Modelling (Unit 2)

4.1 Target, features and evaluation protocol

The target is is_problematic = 1 when the score is 2 or lower; the rating and rating-derived fields are excluded from the predictors. TF-IDF keeps 20,000 unigram and bigram terms (no stop-word list, so negations remain), and truncated SVD reduces the text to 200 components that retain 62.7% of the variance. Nine issue flags, four VADER scores, word count and upvotes add 15 scaled structural features, giving 215 in total.

Logistic Regression and Random Forest [4, 20] use class weights and random state 42. LR uses C = 1; RF uses 150 trees, a 30% sample per tree and a minimum leaf size of 100 to control model size. A stratified 80/20 split gives 910,389 training and 227,598 test reviews. A separate 20% validation split of the training rows checked C and the leaf size; it is not cross-validation.

Table 5. Classifier results on the same 227,598 test reviews

Model	Accuracy	Precision	Recall	PR-AUC
Majority baseline	0.8053	0	0	0.1947
Logistic Regression	0.9216	0.7672	0.8579	0.8710
Random Forest	0.9143	0.7372	0.8701	0.8735

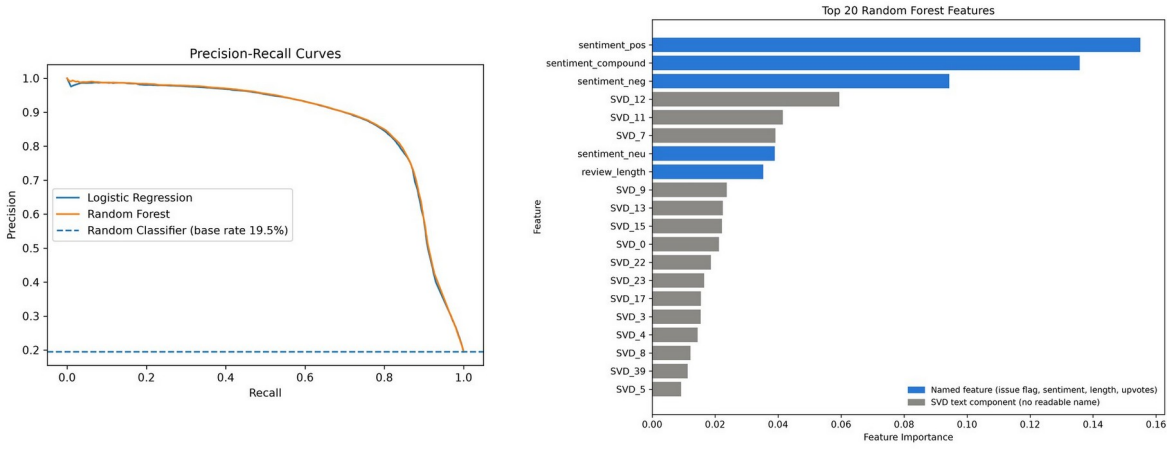


Fig. 4. (a) Precision-recall curves with the 19.47% baseline; (b) the 20 most important RF features, including SVD text components

RF is selected for complaint triage because it recovers 87.0% of low-rating reviews while flagging 23.0% of all reviews. LR produces 16.0% fewer false positives and has higher precision, so it is an alternative when review capacity is limited. The scores support ranking and threshold-based screening; probability calibration was not evaluated separately.

4.2 Leakage audit, errors and subgroup performance

In the initial pipeline, TF-IDF, SVD and scaling were fitted before splitting, which let test reviews influence feature construction. We therefore repeated the 80/20 experiment with all three fitted on training rows only; the results in Table 6 differ from the original by at most 0.0026 across the reported metrics, so the performance conclusion stands.

Table 6. Preprocessing fitted on training rows only (precision and recall at the 0.5 threshold)

Training-only fit	Precision	Recall	PR-AUC
Logistic Regression	0.7698	0.8575	0.8711
Random Forest	0.7364	0.8703	0.8739

The error counts in Table 7 come from the original experiment (preprocessing fitted before splitting), not from the training-only run above, and are shown separately for that reason. The original precision and recall values (Table 5) follow from them.

Table 7. Confusion-matrix counts at the 0.5 threshold in the original experiment (227,598 test reviews)

Original experiment	True negative	False positive	False negative	True positive
Logistic Regression	171,744	11,538	6,297	38,019
Random Forest	169,539	13,743	5,756	38,560

In the original experiment the RF misses 5,756 low-rating test reviews and flags 13,743 higher-rating ones. Some errors reflect short or mis-rated text, and keyword matches also occur in praise, so raising recall creates extra review work. Bootstrap resampling of the held-out predictions (1,000 resamples) gives an RF PR-AUC interval of 0.8708-0.8762; these intervals condition on the fitted model and do not capture every source of collection or training uncertainty.

Table 8. RF performance across domains

Domain	Precision	Recall	PR-AUC
Food & Grocery	0.738	0.905	0.895
Payments	0.625	0.805	0.750
Shopping	0.763	0.844	0.869

Payments has the lowest PR-AUC (0.750; PhonePe 0.678), which supports human review and domain-specific monitoring there. Feature ablation gives an RF PR-AUC of 0.854 for text alone against 0.802 for sentiment alone, and feature importance describes model associations rather than causes. A temporal test (April-August for training, September for testing) gives PR-AUC 0.866; its text representation used all periods, so it should be read alongside the training-only check. Repeated wording and reviewer overlap were not assessed, and the review checker assumes zero upvotes for a new review.

5. Advanced Analytics (Unit 3): Two Selected Methods

5.1 Method 1: text mining with NMF topics and sentiment

Q2 asks what the complaints describe. Cleaning expands contractions, handles negations and lemmatises with NLTK [2]; it retains 167,303 low-rating texts with at least three content words and excludes 54,278 shorter ones. Unigram and bigram TF-IDF matrices (min_df 10, max_df 0.5) are factorised per domain with nndsvda NMF [15], using 6, 6 and 5 topics for food, shopping and payments.

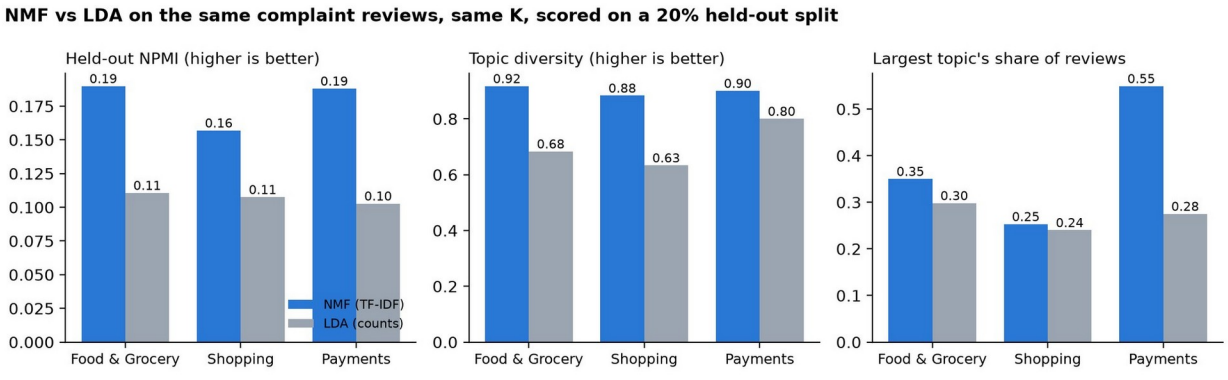


Fig. 5. NMF and LDA comparison at equal K using held-out coherence, topic diversity and held-out topic concentration

NPMI [22] guided the choice of K between 3 and 10. NMF outperforms LDA [3] at equal K: both fit 80% of the documents after full-corpus vocabulary and K selection, so the held-out scores assess these settings, and LDA tuning was limited. Held-out NPMI for NMF is 0.1898, 0.1569 and 0.1880 for food and grocery, shopping and payments, against 0.1106, 0.1075 and 0.1027 for LDA. NMF also has higher topic diversity, meaning fewer repeated terms across topic word lists. These indicators support the representation; they do not measure agreement with human complaint labels. The concentration panel uses held-out assignments, so its 54.9% payments share differs from the 48.6% full-corpus share below.

NMF assigns 167,198 texts, which is 99.94% of eligible complaints and 75.46% of all low ratings; 71,775 of them (42.93%) carry no keyword tag. Assignment coverage is not accuracy, so human labels and language-specific sentiment validation remain necessary.

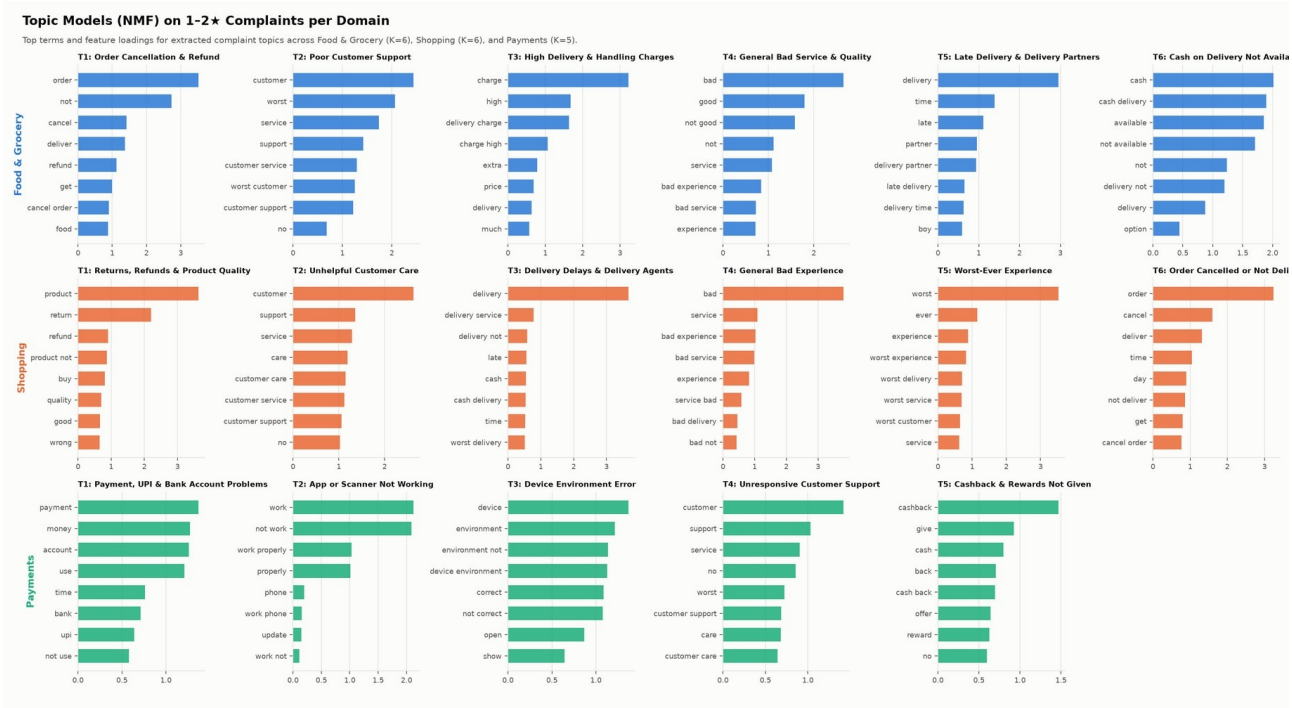


Fig. 6. The 17 NMF topics with their leading terms and component weights, by domain

The leading themes are refunds and cancellations in food (34.7%), returns and product quality in shopping (25.4%) and payment, UPI and bank-account problems in payments (48.6%). Support complaints in food apps average a VADER score of -0.402 [10], while payments app-not-working complaints average -0.093; these are domain-specific text scores. The examples in Table 9 illustrate the topic labels with original spelling retained; they are illustrative, not a random validation sample, and do not estimate assignment accuracy.

Table 9. Representative complaint texts and topic assignments

Domain / app	Review excerpt	Assigned topic	VADER
Food & Grocery / Blinkit	refund history is not clear ,,, 1 mnth back order refund is still waiting	Order Cancellation & Refund	-0.292
Shopping / Myntra	use less product and seller send defect product after that they refuse for return	Returns, Refunds & Product Quality	-0.557
Payments / PhonePe	bank account link nahi ho raha...unable to proceed aa raha he	Payment, UPI & Bank Account Problems	0.000

A refund delay, a refused return and a bank-link failure give the labels an operational meaning. PhonePe receives a neutral VADER score despite describing a failure in mixed Hindi and English, which is why topic and sentiment should be read together.

5.2 Method 2: time-series forecasting and release monitoring

Q3 and Q4 concern expected complaint counts, unusual days and releases. We aggregate 11 daily series over 173 days and fit ARIMA/SARIMAX [11, 23] to the low-rating counts, using a feed-gap input where an app is affected and weekly terms where supported. The training ADF test guides differencing, and ACF/PACF and AICc guide the orders on the first 84 days, favouring fewer terms when AICc values are within two points. Final orders are reselected on all 173 days (Table 10).

Table 10. Selected forecast orders for all 11 apps

App	Backtest order (84 days)	Final order (173 days)	Gap input
Swiggy	(0,1,1) x (1,0,1), 7	(0,0,1) x (1,0,1), 7	Yes
Zomato	(0,1,1) x (1,0,1), 7	(2,1,1) x (1,0,1), 7	No
Blinkit	(0,1,2) x (1,0,1), 7	(1,1,1) x (1,0,1), 7	Yes
Domino's	(0,0,1) x (1,0,1), 7	(0,0,1) x (1,0,1), 7	Yes
Myntra	(0,1,1)	(3,0,2)	No
Flipkart	(0,1,1)	(1,0,1)	Yes
Amazon	(0,0,0)	(1,0,1)	Yes
Meesho	(0,0,0)	(0,0,2)	No
Paytm	(0,1,1)	(2,0,2)	No
PhonePe	(2,0,0)	(1,0,0)	No
Google Pay	(1,0,0)	(1,0,0)	No

In the notation, (p,d,q) are the non-seasonal orders and a second triple gives seasonal orders with a 7-day period. All four food and grocery apps select weekly seasonal terms. The initial Amazon and Meesho models are constant-level (0,0,0), which reflects training-period selection rather than flat future volumes. The backtest keeps its original orders and refits parameters at each origin, so final model selection does not enter the reported backtest errors.

The forecast origin advances weekly across nine origins with a 28-day horizon, and models are refitted at each origin. Seasonal naive repeats the last observed weekday, and Holt-Winters [26] uses a damped additive trend and a weekly additive season. All three are compared on identical rows; overlapping windows create 252 forecasts per app and model, not 252 independent days.

Table 11. Mean forecast accuracy over 11 apps

Model	MAE/day	RMSE/day	MASE	Lower MAE than naive
ARIMA	19.39	26.74	0.934	10/11
Holt-Winters	19.14	26.66	0.933	10/11
Seasonal naive	21.99	31.07	1.080	-

Table 12. Per-app error and interval coverage

App	ARIMA MAE	Naive MAE	ARIMA MASE	95% coverage
Swiggy	28.08	34.63	1.121	92.1%
Flipkart	52.68	56.01	1.249	92.9%
Amazon	12.87	16.94	0.938	85.7%
Paytm	7.59	8.79	0.858	95.2%
Blinkit	28.82	33.82	0.888	97.2%
Meesho	13.63	19.30	0.979	88.5%
Domino's	15.78	18.34	0.837	95.2%
Myntra	12.21	13.91	1.300	88.9%
Zomato	22.78	23.90	0.724	99.2%
Google Pay	10.92	6.19	0.644	100.0%
PhonePe	7.88	10.00	0.736	98.0%

MASE [12] scales each error by the seasonal-naive error on the training data rather than by the held-out MAE. Eight apps have ARIMA MASE below 1 and ten beat seasonal naive on held-out MAE; Google Pay has MASE 0.644 but a worse held-out MAE than naive. ARIMA is retained because it adds prediction intervals and exogenous inputs with accuracy comparable to Holt-Winters. Shopping apps are where it struggles: Flipkart (1.249) and Myntra (1.300) have MASE above 1, and Amazon, Meesho and Myntra under-cover their 95% intervals.

Food and grocery counts on Sundays exceed the app means by 22.4-37.9%, Paytm grows 60.1% between its first and last four-week means, and Swiggy falls 16.2%. These patterns motivated the weekly terms and trend monitoring.

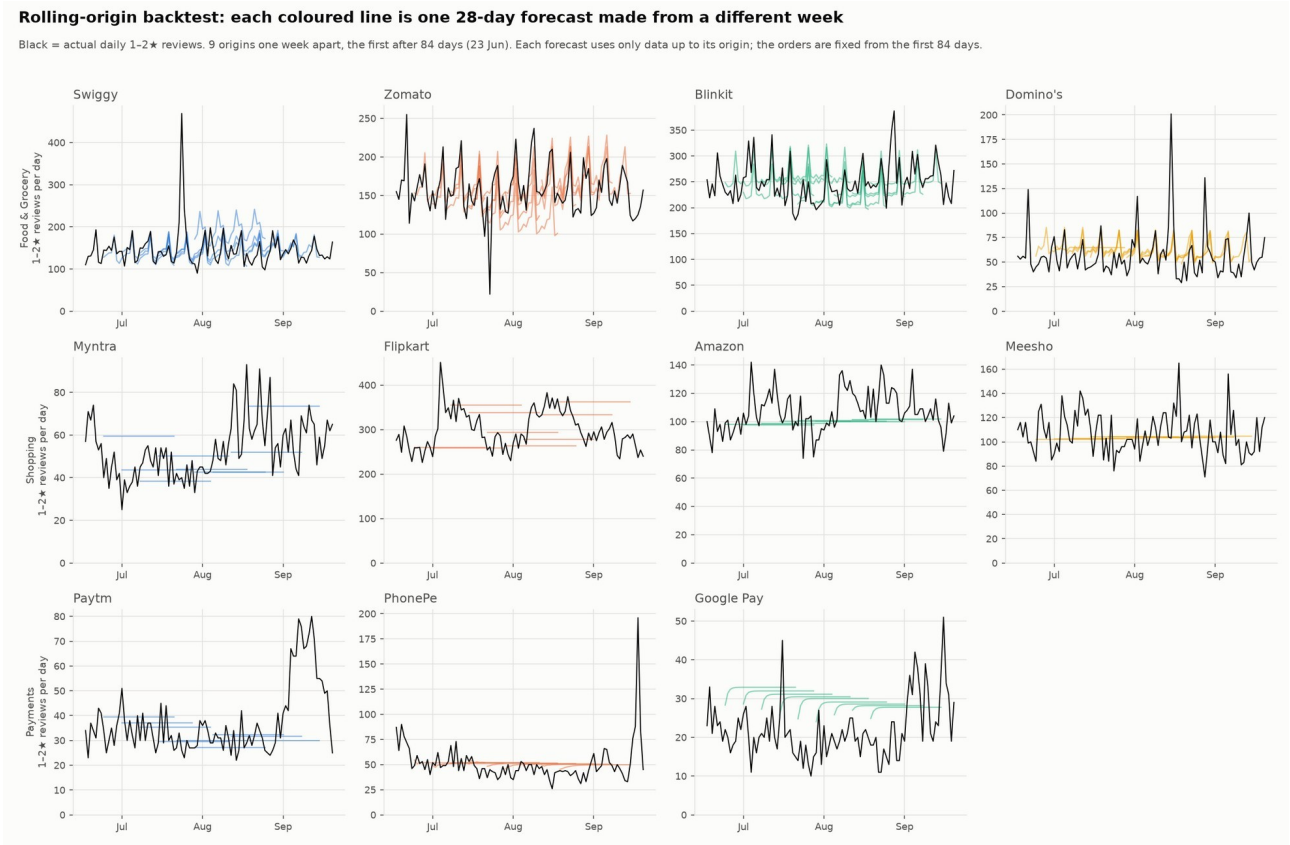


Fig. 7. Rolling backtests for all 11 apps across nine weekly origins with 28-day horizons

Pooled coverage is 82.68% for nominal 80% intervals and 93.90% for 95% intervals, but app-level 95% coverage ranges from 85.71% to 100%, with undercoverage in shopping. The 77 distinct app-days outside the interval (56 above, 21 below) are investigation signals rather than explanations. Final-model Ljung-Box tests at lag 14 do not flag residual autocorrelation at 5% [16], though this does not establish independence. The short history and level shifts limit inference, and the final forecasts for 21 September-18 October 2026 are historical outputs at submission time. Zomato's missing 24 July count is estimated as 165.5 from the same weekday in adjacent weeks; because some donor days lie after certain backtest origins and the estimated day enters scoring, imputation within each training window would be a stricter check.

Release-impact screening.

Release day is inferred from the tenth review on a version, not from an official rollout log. Of 224 events, 205 have adequate new-version and older-version samples and complete windows, and 19 are untestable. The screen compares low-rating shares over the same seven days and adjusts for each app's usual new-version gap using a leave-one-out median. It uses a two-sided z-statistic with Benjamini-Hochberg correction [1]; a deterioration signal needs q below 0.05, an excess gap of at least 5 percentage points and a positive new-minus-older difference. A separate Welch test compares

seven daily counts before and after, and a significant rise corroborates an app-level association. The usual-gap estimate adds uncertainty that the z-statistic ignores, and overlapping releases and self-selected updaters limit any inferential guarantee, so these are exploratory observational screens.

Table 13. The four flagged releases among 205 testable events

App	Version	Excess gap (pp)	App-level corroboration
Swiggy	4.107.1	21.2	No
Swiggy	4.107.2	8.5	No
Flipkart	9.3	7.6	No
Paytm	10.84.5	8.1	Yes

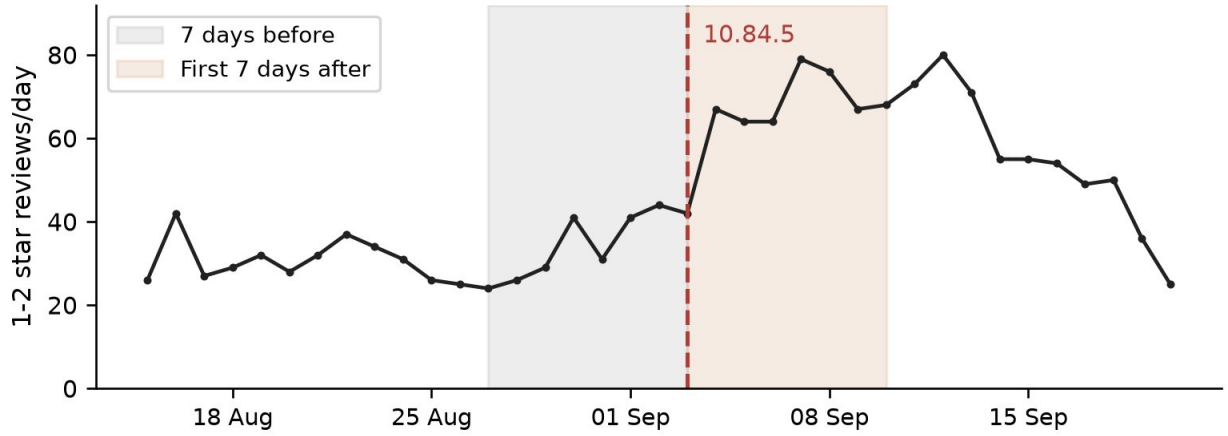


Fig. 8. Paytm low-rating counts around inferred release 10.84.5, with shaded windows for 27 August to 2 September and 3 to 9 September 2026

Of the 205 releases, 196 show no clear change, five show improvement and four show deterioration. No clear change means the screen found no statistically distinguishable difference, which is not evidence that a release had no effect, because the seven-day windows have limited power. Paytm 10.84.5 alone is corroborated at app level, with complaints rising from 33.71 to 65.57 a day in adjacent seven-day windows (q approximately 0.027). The other three flagged releases rest on the new-version comparison only and overlap the feed gap, and same-day controls cannot remove unequal collection bias. Using the first rather than the tenth review as the release day flags the same four releases.

6. Comparative Analysis and Business Impact

6.1 Complementary methods and combined findings

Table 14. Comparison of the two selected Unit 3 methods

Criterion	Text mining	Time series and release screen
Question	What complaint themes and wording occur?	How many complaints, when unusual, and which releases coincide?
Inputs	Eligible low-rating texts; VADER scores	Daily counts, gap flags, inferred release events
Outputs	17 themes, shares, sentiment, examples	Forecasts, intervals, alerts and release flags
Validation	NMF/LDA coherence and diversity comparison	Chronological backtest, baseline errors, coverage
Strength	Finds themes beyond fixed keyword tags	Adds temporal context and volume expectations
Limit	Eligibility exclusions, naming, language and preselected vocabulary	Short history, level shifts, imputation and observational bias

Coherence and MASE measure different tasks and are not ranked against each other. The methods complement Unit 1's service-issue evidence: topics describe what to investigate, while forecasts and release screens indicate timing. Because most releases show no clear change and the tests have limited power, those results cannot show that complaints come from service problems rather than software; that reading rests on the review content alone. The six-signal priority rule combines volume, share, trend, a reliable forecast increase, repeated above-interval days and release flags. It is an interpretable heuristic rather than an optimised causal score.

6.2 Comparison with existing studies

Table 15. Prior-work comparison using comparable scope rather than accuracy rankings

Study	Established contribution	Relationship to this project
Maalej and Nabil [17]	Human-labelled bug, request, experience and rating types; Naive Bayes, trees and MaxEnt	Our target is rating-derived dissatisfaction, so precision and PR-AUC are not directly comparable
AR-Miner [5]	EM-Naive Bayes filtering and LDA grouping; four Android apps	Our internal comparison is NMF against LDA; volume forecasting adds a different output
IDEA [6]	Version-sensitive topic tracking and emerging-issue detection	We add count forecasts, an exploratory release screen and priorities
Khalid et al. [14]	Manual taxonomy of complaint types in low-rated iOS reviews	We derive themes automatically with NMF; most of our tagged complaints concern service rather than app faults
Palomba et al. [19]	Links reviews to code changes; apps that implemented review requests gained ratings	We screen releases from review data alone, without code links, so our release results are exploratory associations
Villarroel et al. [25]	Classifies, clusters and prioritises reviews for release planning	Our six-signal rule prioritises apps from volume, trend, forecast and release evidence rather than single reviews

The studies in Table 15 differ in both their targets and their decision level. Maalej and Nabil [17] classify review types, while our rating-derived target identifies likely dissatisfaction for triage. Our PR-AUC therefore cannot establish verified bug-detection accuracy. AR-Miner [5] and IDEA [6] provide precedents for extracting and tracking issues; our NMF–LDA comparison supports the selected representation on our corpus, and forecasting adds expected complaint volumes. Compared with the manual taxonomy in Khalid et al. [14], our automatically derived themes require human-labelled validation before being treated as reliable complaint classes.

The release and priority results add another distinction. Palomba et al. [19] link review feedback to code changes, whereas our review-based release screen identifies associations for investigation. Villarroel et al. [25] support release planning from individual reviews; our six-signal rule prioritises apps from combined evidence. These differences explain the project's contribution: themes identify what to investigate, forecasts provide a volume expectation, and release evidence indicates when further checks may be useful. They do not demonstrate a causal effect or superior performance over the cited systems.

The supported contribution is integration over a common 173-day dataset, India-relevant service themes and explicit collection-bias checks. The project does not show better accuracy than prior studies, because labels, datasets and protocols differ. Remaining gaps are human-labelled topic validation, multilingual evaluation and causal release analysis.

6.3 KPI-linked business value, priorities and sensitivity scenarios

Measured classifier performance recovers 87.0% of low-rating reviews by reading 23.0% of reviews, and forecasting reduces mean planning error from 21.99 to 19.39 reviews per app per day against seasonal naive. Actual staffing savings, prevented complaints and service improvements have not been observed; the following figures translate the measured outputs into explicitly hypothetical scenarios.

Table 16. Hypothetical triage sensitivity analysis

Scenario	Seconds/read	Cost/hour	Hours saved/day	Net/month
Worst	5	Rs 200	7.0	Rs 40,500
Base	10	Rs 250	14.1	Rs 103,300
Best	20	Rs 300	28.1	Rs 250,700

Assumptions: 6,578 reviews a day across 11 apps, an RF flag rate of 0.229804, 30 operating days a month and 8.7 maintenance hours a month at the same hourly cost. About 5,066 reads a day are skipped, and hours saved equal skipped reads times seconds per read divided by 3,600. Net value is the gross value of saved hours minus the maintenance cost, rounded to the nearest Rs 100, and about

167 low-rating reviews a day would be missed. No monetary cost of missed complaints or model mistakes is included, so this is an effort-value scenario, not realised profit.

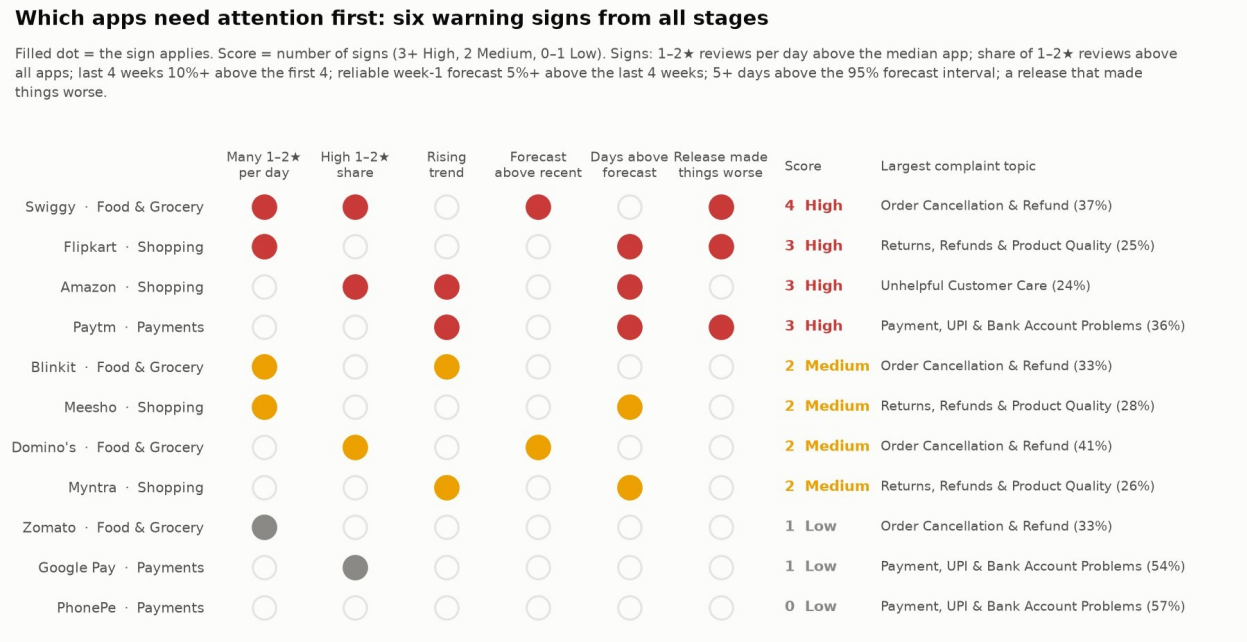


Fig. 9. App priorities showing the six warning signs, scores and the leading complaint topic for each app

Each warning sign contributes one point: three or more is High, two Medium, and zero or one Low. These equal-weight operational rules have not been optimised against intervention outcomes. Swiggy, Flipkart, Amazon and Paytm are High, four apps are Medium and three are Low. The signs are: daily volume above the cross-app median (about 103.0 a day); low-rating share above the pooled share (about 18.94%, feed-gap days excluded); a last four-week mean at least 10% above the first; a reliable forecast increase (week 1 at least 5% above the recent mean, a lower ARIMA MAE than naive and a change of at most 30%); at least five distinct days above the 95% backtest interval; and at least one deterioration flag from the release screen.

For Paytm, daily volume (37.8 a day) and share (14.64%) do not trigger, but rising volume (+60.1%), 10 above-interval days and the flagged release 10.84.5 do, so it is High with three points. Its historical week-1 forecast of 136 complaints against a recent weekly mean of 348.2 (-61.0%) breaks the 30% change guard and is excluded, and gives no basis for reducing monitoring. Two Paytm windows should not be confused: the seven-day release test gives 33.71 to 65.57 a day, whereas the full 18 post-release days (3-20 September) average 59.72 a day, 26.01 above the same baseline or 182.06 a week. That arithmetic does not establish causation, and any avoided-review benefit would depend on a fix removing the excess.

Text mining supplies the issues to investigate: payment, UPI and bank-account problems make up 35.77% of 4,809 assigned Paytm complaints, and cashback or reward problems 17.51% (mean

VADER -0.233). These full-period app shares differ from the domain shares in Section 5.1 and do not show which issue caused the release-associated rise; Unit 2 precision for Paytm is 66.1%, which supports manual checking. In order of impact and feasibility, we prioritise (1) triage for food and shopping, (2) a daily release screen with a volume alert, (3) fixes for Swiggy refunds and Amazon customer care, and (4) payments triage with human review (Section 9).

7. Interactive Application, Figure Index and Reproducibility

7.1 Application architecture and working interactions

The six-page Streamlit application [24] reads precomputed summaries and saved classifier and topic models. Overview integrates Unit 1, Check a Review demonstrates Unit 2, Text mining, Forecasting and Update impact expose Unit 3, and Recommendations joins the stages. Plotly renders the interactive charts [21]. The public application is at https://team10-app-reviews.streamlit.app/ and can also be launched locally with the command streamlit run dashboard/app.py from the repository root. Filtering, aggregation and the review checker are computed live, whereas forecasts, topics, release tests and priorities are precomputed, so changing a selector does not retrain a model.

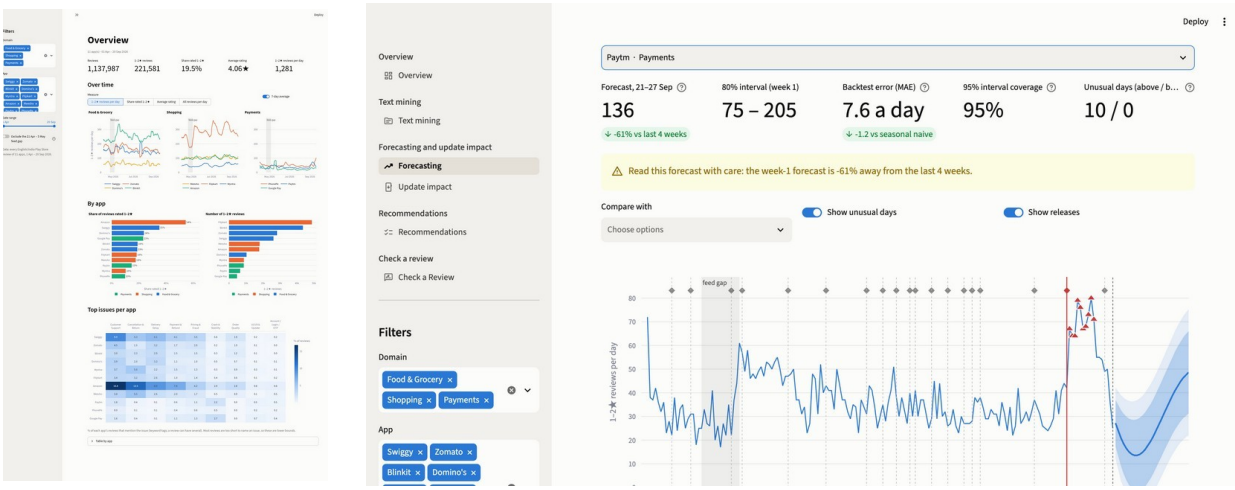


Fig. 10. Application screenshots showing (a) the Overview main state and (b) the Forecasting drill-down for Paytm

7.2 Figure index and reproduction map

Table 17 maps every figure to the section where it is discussed and the script that produces it.

Table 17. Figure index and reproduction sources

Figure / section	Title and source
1 / 1.1	Workflow: newformat/figures/make_workflow_diagram.py (Matplotlib [9])
2-3 / 3	Within-domain comparisons and feed gap: scripts/05_eda.py (Matplotlib)
4 / 4.1	PR curves and feature importance: scripts/07_model_evaluation_charts.py, scripts/08_model_interpretation.py
5-6 / 5.1	NMF/LDA comparison and 17 topic panels: scripts/23_topic_model_comparison.py, scripts/17_topic_modelling.py
7-8 / 5.2	Rolling backtests: scripts/19_forecasting.py; Paytm release windows: newformat/figures/make_final_report_figures.py

Figure / section	Title and source
9 / 6.3	Six warning signs and priorities: scripts/22_combined_insights.py
10 / 7.1	Overview and Paytm Forecasting: dashboard/screenshots/ (Streamlit/Plotly)
D1 / Appendix D	Project-board snapshot: docs/Review_2_Board.png (GitHub Projects)

7.3 Code and environment

The repository (https://github.com/Adithya-Monish-Kumar-K/ba-capstone-app-reviews) holds the scripts, data, models, notebooks and dashboard, with a README, a requirements file (Python 3.13 or later; scikit-learn 1.9.0 to match the saved models) and run instructions in Appendix C. Re-scraping a changing store does not reproduce the reference snapshot, so the committed data should be reused. Local verification on 8 November 2026 ran both notebooks, refitted all 11 backtests, repeated the topic and release analyses and checked aggregates. All 18 page states, 24 sidebar interactions and 30 edge cases passed after a zero-review Overview guard was corrected, and results matched rounding or documented retraining variation.

Overview uses the app, date and gap filters, Forecasting filters history while keeping a fixed prediction horizon, topic totals and recommendations use full-period outputs, and release comparisons keep their original windows. The first review-checker load took about 24 seconds locally. These were local tests: response time on the hosted application was not measured, so the under-2-second target remains unverified and needs cold-start and hosted testing.

8. End-to-End Validation, Traceability and Contributions

8.1 Objective outcomes and validation scope

The declared scope is implemented across Units 1 to 3 and the application. Threats to validity remain: a low rating measures dissatisfaction rather than a confirmed defect, the feed gap and self-selected updaters can bias shares, the sample covers 11 large Indian apps over 173 days of Latin-script Android reviews, and re-scraping cannot reproduce the exact snapshot.

Table 18. Objective-to-evidence traceability and qualified outcomes

Objective	Evidence and outcome	Action
O1	Section 4.2: audit PR-AUC 0.8739; recall 0.8703; met	R4
O2	Section 5.1: 99.94% of eligible complaints themed; NMF above LDA; met	R1
O3	Section 5.2: 8/11 apps with MASE < 1; 93.9% coverage; met, with app-level limits	R3
O4	Section 5.2: 205 tested; four flagged; exploratory	R2
O5	Section 6.3: 11 priorities; 65 recommendations; met	R1-R5
O6	Sections 7.1 and 7.3: dashboard live and locally verified; hosted response time not measured; partly met	R5

8.2 Project tracking and individual contributions

The project was tracked on a GitHub Projects board (https://github.com/users/Adithya-Monish-Kumar-K/projects/4) with the stages Backlog, To Do, In Progress, Review/Testing and Completed.

Review 1 had 15 tasks and Review 2 had 14; all 29 are completed and each is covered by a merged pull request. Work ran as a relay in which each stage started from the merged output of the previous one, and the board snapshot is in Appendix D. Table 19 lists each member's work and evidence; individual marks may vary with participation and explanation.

Table 19. Individual contributions (R1 = Review 1, R2 = Review 2)

Member and register number	Individual contribution	Evidence (issues, pull requests, files)
Aditya Monish Kumar K / CB.SC.U4CSE23103	R1: owned data collection and preprocessing and set up the project board. Wrote the Play Store scraper, collected the raw reviews for the 11 apps, built the cleaning pipeline (duplicates, empty and letter-free text, language filter, derived fields) and the data-integrity tests. R2 (Stage 1 of the relay): built the daily and weekly time series, release events and STL/ADF/ACF-PACF diagnostics; the text preprocessing and document-term matrices; and the dashboard skeleton with shared filters, the Overview page and the public deployment.	R1 issues #53, #54, #56-#58; PR #52, #99. R2 issues #131, #132, #133; PR #147, #148, #149. scripts/01_scrape_reviews.py, scripts/02_clean_reviews.py, scripts/15_time_series_data.py, scripts/16_text_preprocessing.py, dashboard/app.py, dashboard/pages/overview.py
Akshay KS / CB.SC.U4CSE23104	R1: exploratory data analysis, shared with Regella Krishna Saketh (EDA script and 15 charts, per-app and length breakdowns, coverage and bias audit, issue analysis, results write-up). R2 (Stage 3): ARIMA/SARIMAX forecasting with the rolling-origin backtest, seasonal-naive and Holt-Winters baselines and the four-week forecast; built the Text-mining dashboard page with example reviews; and made the final fixes (training-only differencing, per-origin MASE, text cleaner).	R1 issues #104-#106, #118-#120 (shared); PR #121. R2 issues #135, #139; PR #152, #153, #162. scripts/05_eda.py, scripts/19_forecasting.py, scripts/20_topic_examples.py, dashboard/pages/text_mining.py
Kanishka D / CB.SC.U4CSE23155	R1: feature engineering and predictive models, shared with Harshini Vennela (TF-IDF and SVD feature pipeline, Logistic Regression and Random Forest, evaluation charts, results write-up). R2 (Stage 5): release-impact tests (before/after and new-versus-older) for the 224 release events; the combined priority table and 65 recommendations; the Forecasting, Update-impact and Recommendations pages; and the final Review 2 notebook.	R1 issues #107-#110 (shared); PR #122-#125. R2 issues #138, #140, #141, #143; PR #157, #158. scripts/05_feature_engineering.py, scripts/06_model_training.py, scripts/21_update_impact.py, scripts/22_combined_insights.py, dashboard/pages/forecasting.py, notebooks/Review_2.ipynb
Harshini Vennela / CB.SC.U4CSE23455	R1: feature engineering and predictive models, shared with Kanishka D (feature pipeline, the two classifiers, evaluation and write-up). R2 (Stage 4): forecast evaluation (MAE, RMSE, MASE, interval coverage, Ljung-Box residual checks and unusual-day alerts), the forecast write-up, and the Check-a-review dashboard page.	R1 issues #107-#110 (shared); PR #122-#125. R2 issues #137, #142, #146; PR #154, #155, #156. scripts/05_feature_engineering.py, scripts/06_model_training.py, scripts/20_forecast_evaluation.py, dashboard/pages/check_review.py
Regella Krishna Saketh / CB.SC.U4CSE23649	R1: exploratory data analysis, shared with Akshay KS (coverage and bias audit, issue analysis, charts and write-up). R2 (Stage 2): text mining as Method 1, covering the NMF topic models, distinctive terms per app, sentiment per topic and the praise contrast, followed by topic coherence, comparison with the keyword tags, untagged reviews, topic trends and the write-up.	R1 issues #104-#106, #118-#120 (shared); PR #121. R2 issues #134, #136; PR #150, #151. scripts/05_eda.py, scripts/17_topic_modelling.py, scripts/18_topic_evaluation.py, TEXT_MINING.md

Collaborative work: Akshay KS and Regella Krishna Saketh shared the Review 1 exploratory analysis (one pull request, six issues), and Harshini Vennela and Kanishka D shared the Review 1

feature engineering and model work. In Review 2 the relay was sequential: Aditya Monish Kumar K prepared the data for both methods, Regella Krishna Saketh built the topic models, Akshay KS the forecasts, Harshini Vennela the forecast evaluation and Kanishka D the release tests and combined insights.

8.3 AI use and originality declaration

Claude Code (Anthropic Claude) and OpenAI Codex assisted with drafting, revising and formatting documentation, including this report, and with code review, plots, the final audit and targeted application corrections. Except where sources are cited, the ideas, analysis and conclusions belong to the team, and any text or code drafted with AI assistance was checked, edited and verified by the team before it was used. AI does not replace the team's responsibility for the work.

9. Conclusion, Recommendations and Roadmap

The workflow delivers rating-based triage, complaint themes, volume forecasts, release screening and app priorities. Table 18 links each objective to its evidence: five are met (O4 as an exploratory screen) and O6 is partly met because hosted response time was not measured. The roadmap turns the findings into support, product and quality-assurance actions, with monitored pilots to measure operational and financial outcomes.

Table 20. Prioritised roadmap with owners and success measures

When / owner	Action and success check
Immediate / Support and Product	R1 Improve Swiggy refunds and Amazon customer care; track response time and low-rating share. A five-point reduction is a proposed target.
Immediate / Paytm QA	R2 Investigate 10.84.5 and cashback complaints with logs; compare post-fix counts with forecasts and a control period. The pre-release mean of about 34 a day is a proposed reference, not a guaranteed recovery.
Within 1 month / Support and Data	R3 Forecast staffing and Flipkart alerts; R4 pilot reviewed triage, especially for Payments. Monitor errors and coverage.
1-3 months / QA and Data	R5 Automate release screens; label 1,000 reviews; validate training-window preprocessing; measure cold-start and hosted latency.
3-12 months / Product and Engineering	Connect ticketing; collect a full year of data; evaluate multilingual models and controlled interventions (future work).

References

[1] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300.

[2] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media, Sebastopol, CA.

[3] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *Journal of Machine Learning Research* 3 (2003), 993–1022.

[4] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32.

[5] Ning Chen, Jialiu Lin, Steven C. H. Hoi, Xiaokui Xiao, and Boshen Zhang. 2014. AR-Miner: Mining informative reviews for developers from mobile app marketplace. In *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. ACM, 767–778.

[6] Cuiyun Gao, Jichuan Zeng, Michael R. Lyu, and Irwin King. 2018. Online app review analysis for identifying emerging issues. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. ACM, 48–58.

[7] Government of India. 2023. The Digital Personal Data Protection Act, 2023 (No. 22 of 2023). Ministry of Law and Justice, New Delhi.

[8] Emitza Guzman and Walid Maalej. 2014. How do users like this feature? A fine grained sentiment analysis of app reviews. In *Proceedings of the 22nd IEEE International Requirements Engineering Conference (RE '14)*. IEEE, 153–162.

[9] John D. Hunter. 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.

[10] C. J. Hutto and Eric Gilbert. 2014. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM '14)*. AAAI, 216–225.

[11] Rob J. Hyndman and George Athanasopoulos. 2021. *Forecasting: Principles and Practice* (3rd ed.). OTexts, Melbourne. https://otexts.com/fpp3/

[12] Rob J. Hyndman and Anne B. Koehler. 2006. Another look at measures of forecast accuracy. *International Journal of Forecasting* 22, 4 (2006), 679–688.

[13] JoMingyu. [n. d.]. google-play-scraper: Google Play Store scraper for Python (software package). Retrieved 2026 from https://pypi.org/project/google-play-scraper/

[14] Hammad Khalid, Emad Shihab, Meiyappan Nagappan, and Ahmed E. Hassan. 2015. What do mobile app users complain about? *IEEE Software* 32, 3 (2015), 70–77.

[15] Daniel D. Lee and H. Sebastian Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature* 401 (1999), 788–791.

[16] Greta M. Ljung and George E. P. Box. 1978. On a measure of lack of fit in time series models. *Biometrika* 65, 2 (1978), 297–303.

[17] Walid Maalej and Hadeer Nabil. 2015. Bug report, feature request, or simply praise? On automatically classifying app reviews. In *Proceedings of the 23rd IEEE International Requirements Engineering Conference (RE '15)*. IEEE, 116–125.

[18] Dennis Pagano and Walid Maalej. 2013. User feedback in the AppStore: An empirical study. In *Proceedings of the 21st IEEE International Requirements Engineering Conference (RE '13)*. IEEE, 125–134.

[19] Fabio Palomba, Mario Linares-Vásquez, Gabriele Bavota, Rocco Oliveto, Massimiliano Di Penta, Denys Poshyvanyk, and Andrea De Lucia. 2015. User reviews matter! Tracking crowdsourced reviews to support evolution of successful apps. In *Proceedings of the 31st IEEE International Conference on Software Maintenance and Evolution (ICSME '15)*. IEEE, 291–300.

[20] Fabian Pedregosa et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[21] Plotly Technologies Inc. 2015. Collaborative data science. Montréal, QC. https://plot.ly

[22] Michael Röder, Andreas Both, and Alexander Hinneburg. 2015. Exploring the space of topic coherence measures. In *Proceedings of the 8th ACM International Conference on Web Search and Data Mining (WSDM '15)*. ACM, 399–408.

[23] Skipper Seabold and Josef Perktold. 2010. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*. 92–96.

[24] Streamlit Inc. [n. d.]. Streamlit: A faster way to build and share data apps (software). Retrieved 2026 from https://streamlit.io

[25] Lorenzo Villarroel, Gabriele Bavota, Barbara Russo, Rocco Oliveto, and Massimiliano Di Penta. 2016. Release planning of mobile apps based on user reviews. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. ACM, 14–24.

[26] Peter R. Winters. 1960. Forecasting sales by exponentially weighted moving averages. *Management Science* 6, 3 (1960), 324–342.

Appendix A: Data Dictionary and Validation

A.1 Complete final-data dictionary

The validated snapshot contains 1,137,987 review records in 11 per-app gzipped CSV files and 29 columns. The dictionary below records every final field. Review IDs support deduplication; neither storefront settings nor a retained review ID establish a reviewer's physical location or identity.

Table A1. Complete dictionary of all 29 final fields

Field	Type	Meaning
app_id	category	Play Store package name
app_name	category	One of the 11 apps
domain	category	Food & Grocery, Shopping or Payments
review_id	text	Play Store's unique review id
score	integer	Star rating given by the user (1–5)
content	text	Review text
thumbs_up	integer	Users who marked the review helpful
app_version	category	App version on the reviewer's phone; empty when the Play Store gives none
review_date	datetime	Date and time the review was posted
month	category	Year-month of review_date
review_length	integer	Number of words in content
issue_crash_bugs_stability	integer (0/1)	1 if the text matches the Crash & Stability keywords
issue_payment_refund	integer (0/1)	1 if the text matches the Payment & Refund keywords
issue_delivery_delay	integer (0/1)	1 if the text matches the Delivery Delay keywords
issue_order_quality_fulfillment	integer (0/1)	1 if the text matches the Order Quality & Fulfilment keywords
issue_cancellation_return	integer (0/1)	1 if the text matches the Cancellation & Return keywords
issue_customer_support	integer (0/1)	1 if the text matches the Customer Support keywords
issue_account_login_otp	integer (0/1)	1 if the text matches the Account / Login / OTP keywords
issue_pricing_charges_fraud	integer (0/1)	1 if the text matches the Pricing, Charges & Fraud keywords
issue_ui_ux_update	integer (0/1)	1 if the text matches the UI/UX & Update keywords
issue_count	integer	Number of the 9 issue tags the review matches
has_issue	integer (0/1)	1 if issue_count > 0
primary_issue	category	Tag with the most keyword matches, or 'none'
all_issues	text	All matched tags joined by ';', or 'none'
sentiment_neg	float	VADER share of negative words (0–1)
sentiment_neu	float	VADER share of neutral words (0–1)
sentiment_pos	float	VADER share of positive words (0–1)
sentiment_compound	float	VADER overall score from −1 (very negative) to +1 (very positive)
sentiment_label	category	Positive if compound ≥ 0.05, Negative if ≤ −0.05, else Neutral

Source: DATA_DICTIONARY.md; data/sample/final_dataset_sample.csv (110 rows, ten per app).

A.2 Recorded validation and treatment of exceptions

The saved validation record contains all 18 passed checks. Documented exceptions and their treatment are retained below. These checks establish consistency of the stored dataset; sampling and modelling limitations are discussed in the main report.

Table A2. All 18 validation checks and their recorded outcomes

No.	Check passed	Recorded evidence or exception
1	Row count matches the cleaning log	1,137,987 tagged rows, 1,137,987 cleaned rows, 1,137,987 in cleaning_results.json
2	Same reviews in the cleaned and tagged files	review_id sets are identical
3	No duplicate reviews	0 duplicate review_id
4	No missing values outside app_version	0 missing in the other 28 columns
5	app_version missing share is as documented	152,017 missing (13.4%); kept, not used as a model input
6	Ratings are 1–5	min 1, max 5
7	Helpful votes are not negative	min 0, max 11,165
8	Every review is inside 1 Apr – 20 Sep 2026	01 Apr 2026 00:01 to 20 Sep 2026 23:59
9	month agrees with review_date	all rows
10	review_length is the word count of content	all rows
11	Every review has letters	emoji-only and punctuation-only reviews were removed in cleaning
12	Domain matches the app	per scripts/apps.py
13	issue_count equals the sum of the 9 issue flags	all rows
14	has_issue agrees with issue_count	all rows
15	primary_issue is 'none' exactly when no tag matched	all rows
16	VADER scores are in range and neg + neu + pos ≈ 1	compound -0.999 to 1.000; sums within 0.01 of 1, except 47 reviews made only of one-letter tokens ("o k"), which VADER scores 0 on all four (Neutral)
17	sentiment_label follows the compound thresholds	±0.05
18	Daily coverage	173 of 173 days for 10 apps; Zomato 172 (24 Jul missing in the Play Store itself)

Cleaning retained 1,137,987 of 1,202,729 collected rows: 47,533 empty or very short texts, 7,420 texts without letters and 9,789 below the Latin-letter threshold were removed; duplicate review IDs removed: zero. App-version missingness remains documented. The Latin-letter rule is a script filter, so mixed Hindi–English text can remain. No blanket licence or complete anonymisation of raw text is asserted.

Source: data/dataset_validation.json; DATA_VALIDATION.md; data/cleaning_results.json; DATA_SOURCES.md.

Appendix B: Supplementary Evaluation

B.1 Classifier uncertainty features and app-level performance

These tables describe the original held-out experiment. The separate training-only preprocessing audit is reported in Section 4.2. Bootstrap intervals condition on the fitted model and held-out predictions.

Table B1. Bootstrap intervals from 1,000 resamples

Metric	LR estimate [95% CI]	RF estimate [95% CI]
precision	0.7672 [0.7636, 0.7709]	0.7372 [0.7338, 0.7410]
recall	0.8579 [0.8547, 0.8609]	0.8701 [0.8670, 0.8730]
pr_auc	0.8710 [0.8681, 0.8738]	0.8735 [0.8708, 0.8762]

Table B2. Feature ablation on the original reference split

Feature set	Count	LR PR-AUC	RF PR-AUC
text only	200	0.8524	0.8542
sentiment only	4	0.7390	0.8016
structural only	15	0.8395	0.8482
all except sentiment	211	0.8573	0.8569
all features	215	0.8710	0.8735

Table B3. RF held-out performance for all 11 apps

App	Test reviews	Precision	Recall	PR-AUC
Amazon	6,705	0.908	0.961	0.969
Blinkit	45,707	0.664	0.906	0.865
Domino's	8,568	0.783	0.889	0.898
Flipkart	53,192	0.771	0.742	0.800
Google Pay	4,049	0.685	0.855	0.813
Meesho	20,509	0.693	0.942	0.909
Myntra	16,954	0.654	0.949	0.926
Paytm	9,026	0.661	0.840	0.798
PhonePe	17,672	0.569	0.751	0.678
Swiggy	15,001	0.873	0.935	0.948
Zomato	30,215	0.739	0.881	0.879

Implementation settings: classifier TF-IDF uses min_df=3, max_df=0.95, sublinear_tf=True and up to 20,000 unigram/bigram terms; SVD has 200 components. LR uses max_iter=1000. Validation checked LR C=0.01, 0.1, 1, 10 and RF minimum leaf sizes 25, 50, 100, 200. No probability-calibration claim is made.

Source: data/model_bootstrap_ci.csv; model_ablation.csv; model_results_by_group.csv; model_tuning.csv; scripts/05_feature_engineering.py, 06_model_training.py, 13_model_robustness_checks.py.

B.2 Complete topic frequencies and sentiment

NMF assigns 167,198 of 167,303 eligible texts; 105 remain unassigned. Eligible texts have at least three content words. The assignment rate is 99.94% of eligible complaints and 75.46% of all low-rating reviews. Of assigned texts, 71,775 (42.93%) carry no keyword tag. Topic labels are interpretations of term patterns, not human-verified complaint classes.

Table B4. All 17 topics with within-domain shares and mean sentiment

Domain	Topic label	Reviews	Share	VADER mean
Food & Grocery	1 Order Cancellation & Refund	28,653	34.7%	-0.272
Food & Grocery	2 Poor Customer Support	15,641	18.9%	-0.402
Food & Grocery	3 High Delivery & Handling Charges	8,724	10.6%	-0.214
Food & Grocery	4 General Bad Service & Quality	10,394	12.6%	-0.437
Food & Grocery	5 Late Delivery & Delivery Partners	14,861	18.0%	-0.295
Food & Grocery	6 Cash on Delivery Not Available	4,304	5.2%	-0.116
Shopping	1 Returns, Refunds & Product Quality	18,123	25.4%	-0.229
Shopping	2 Unhelpful Customer Care	13,120	18.4%	-0.243
Shopping	3 Delivery Delays & Delivery Agents	11,150	15.6%	-0.295
Shopping	4 General Bad Experience	6,195	8.7%	-0.595
Shopping	5 Worst-Ever Experience	6,976	9.8%	-0.613
Shopping	6 Order Cancelled or Not Delivered	15,812	22.2%	-0.310
Payments	1 Payment, UPI & Bank Account Problems	6,443	48.6%	-0.145
Payments	2 App or Scanner Not Working	1,305	9.9%	-0.093
Payments	3 Device Environment Error	1,146	8.7%	-0.129
Payments	4 Unresponsive Customer Support	2,410	18.2%	-0.229
Payments	5 Cashback & Rewards Not Given	1,941	14.7%	-0.150

Shares use each domain's topic-assigned corpus: Food and Grocery 82,577; Shopping 71,376; Payments 13,245. They are not percentages of all reviews or all users. NMF uses nndsvda initialisation and max_iter=250 with 6, 6 and 5 topics. Figure 6 displays component term weights; this table displays review frequencies. VADER is an English lexicon score; mixed-language complaint examples can receive neutral scores despite describing a failure.

The NMF–LDA comparison in Figure 5 uses models fitted on 80% of documents after full-corpus vocabulary and K selection. Its held-out concentration panel therefore has a different denominator and fitted model from this full-corpus table. Neither automatic assignment coverage nor coherence establishes label accuracy.

Source: data/textmining/topic_sentiment.csv; topic_model_comparison.csv; topic_examples.csv; scripts/17_topic_modelling.py, 18_topic_evaluation.py, 23_topic_model_comparison.py.

Appendix C: Repository Environment and Run Instructions

C.1 Installation and exact analysis commands

Run from the repository root using Python 3.13 or later. requirements.txt defines the analysis dependencies; scikit-learn 1.9.0 matches the saved classifier models. The committed raw, clean and tagged snapshots support reproduction without re-querying a changing storefront. Re-scraping is optional and will not reproduce an unchanged live source. Training and the preprocessing audit require substantial memory and time.

```
python -m pip install -r requirements.txt
python -m pip install ipython ipykernel jupyterlab
python -m nltk.downloader vader_lexicon wordnet stopwords
python scripts/02_clean_reviews.py
python scripts/03_tag_issues.py
python scripts/04_sentiment_analysis.py
python scripts/05_eda.py
python scripts/05_feature_engineering.py
python scripts/06_model_training.py
python scripts/07_model_evaluation_charts.py
python scripts/08_model_interpretation.py
python scripts/13_model_robustness_checks.py
python scripts/15_time_series_data.py
python scripts/16_text_preprocessing.py
python scripts/17_topic_modelling.py
python scripts/18_topic_evaluation.py
python scripts/20_topic_examples.py
python scripts/19_forecasting.py
python scripts/20_forecast_evaluation.py
python scripts/21_update_impact.py
python scripts/22_combined_insights.py
python scripts/23_topic_model_comparison.py
python scripts/24_dataset_documentation.py
python scripts/25_train_only_preprocessing_check.py
streamlit run dashboard/app.py
```

Notebook alternatives: jupyter lab notebooks/Review_1_Analysis.ipynb and jupyter lab notebooks/Review_2.ipynb document the same analysis sequence. To collect a fresh source snapshot, run python scripts/01_scrape_reviews.py separately. The report formatter reads existing results and does not run any training stage.

Source: requirements.txt; notebooks/Review_1_Analysis.ipynb; notebooks/Review_2.ipynb; repository README.md.

C.2 Dashboard guide and interpretation of interactions

Public application: https://team10-app-reviews.streamlit.app/. Local launch: streamlit run dashboard/app.py, then open the local address printed by Streamlit. The six pages combine saved analytical outputs with live filtering, aggregation and the review checker. Changing a selector does not refit topic models, forecasts, release tests or priorities.

Table C1. User guide to all six application pages

Page	User action	Scope and interpretation
Overview	Select apps/domains and dates; compare counts, low-rating shares and issues.	Date and feed-gap filters affect the review aggregates. Check the scope captions before comparing denominators.
Check a Review	Paste a review and inspect the predicted triage flag and supporting fields.	The saved classifier computes a live prediction. New-review upvotes are assumed to be zero; the score is not separately calibrated.
Text mining	Choose a domain/app and inspect themes, sentiment, examples and topic trends.	Topic totals and examples use full-period assignments; date controls limit the trend view, not every displayed total.
Forecasting	Choose an app; compare forecasts, baselines, intervals, alerts and release markers.	The history filter changes displayed observations; the saved historical forecast horizon remains fixed.
Update impact	Select an app/release and compare new/older-version shares and count windows.	Each comparison keeps its original release window. Deterioration flags are observational screens, not causal attribution.
Recommendations	Inspect priorities and evidence, filter the actions and export recommendations.	Scores and recommendations come from the full-period combined analysis and the six documented rules.

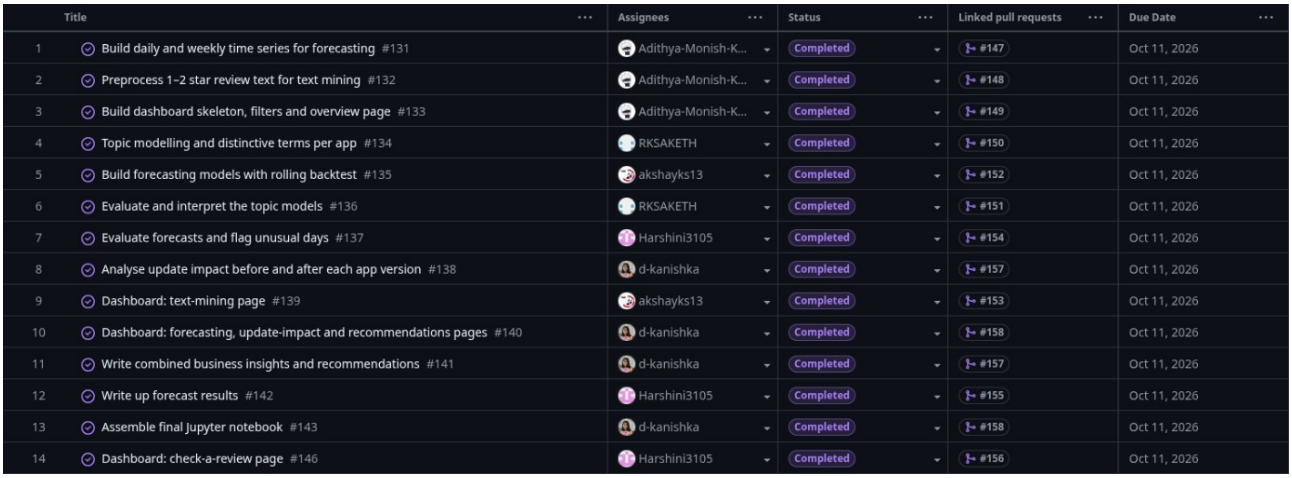
Local checks recorded on 8 November 2026 covered 18 page states, 24 sidebar interactions and 30 edge cases. Empty selections and zero-review days return messages after the Overview guard was corrected. The first checker load took about 24 seconds locally; hosted latency was not measured, so the under-two-second target is not claimed as met.

Artifact map: data/tagged/ holds final reviews; data/textmining/ holds assignments, topics and comparison scores; data/timeseries/ holds orders, backtests, interval coverage and alerts; data/update_impact/ holds release tests; data/combined/ holds priorities and recommendations. Saved models are in models/, and application pages are in dashboard/pages/.

Source: dashboard/app.py; dashboard/pages/; README.md; TEXT_MINING.md; TIME_SERIES.md; COMBINED_INSIGHTS.md.

Appendix D: Tracking Evidence

GitHub Projects: https://github.com/users/Adithya-Monish-Kumar-K/projects/4. The project used Backlog, To Do, In Progress, Review/Testing and Completed. The retained Review 2 snapshot shows 14 assigned tasks, due dates, completed states and linked pull requests; it is a recorded snapshot rather than live board status.



	Title	Assignees	Status	Linked pull requests	Due Date
1	Build daily and weekly time series for forecasting #131	Adithya-Monish-K...	Completed	#147	Oct 11, 2026
2	Preprocess 1–2 star review text for text mining #132	Adithya-Monish-K...	Completed	#148	Oct 11, 2026
3	Build dashboard skeleton, filters and overview page #133	Adithya-Monish-K...	Completed	#149	Oct 11, 2026
4	Topic modelling and distinctive terms per app #134	RKSAKETH	Completed	#150	Oct 11, 2026
5	Build forecasting models with rolling backtest #135	akshayks13	Completed	#152	Oct 11, 2026
6	Evaluate and interpret the topic models #136	RKSAKETH	Completed	#151	Oct 11, 2026
7	Evaluate forecasts and flag unusual days #137	Harshini3105	Completed	#154	Oct 11, 2026
8	Analyse update impact before and after each app version #138	d-kanishka	Completed	#157	Oct 11, 2026
9	Dashboard: text-mining page #139	akshayks13	Completed	#153	Oct 11, 2026
10	Dashboard: forecasting, update-impact and recommendations pages #140	d-kanishka	Completed	#158	Oct 11, 2026
11	Write combined business insights and recommendations #141	d-kanishka	Completed	#157	Oct 11, 2026
12	Write up forecast results #142	Harshini3105	Completed	#155	Oct 11, 2026
13	Assemble final Jupyter notebook #143	d-kanishka	Completed	#158	Oct 11, 2026
14	Dashboard: check-a-review page #146	Harshini3105	Completed	#156	Oct 11, 2026

Fig. D1. Review 2 project board with task owners due dates and linked pull requests

Source: docs/Review_2_Board.png; GitHub Projects board above.